SCHOOL MANAGEMENT SYSTEM

Software Requirements Specification (SRS)
& Complete Design Documentation

Document Version:	1.0 (Final)
Status:	Approved for Development
Date:	13 November 2026
System Name:	School Management System
Includes Modules:	Students, Teachers, Classes, Attendance, Exams, Fees, Lost & Found, Notifications

This document provides the complete requirements, system architecture, database design, API specification, UML descriptions, and UI guidelines for the School Management System.

1. Table of Contents

1. Table of Contents
2. Introduction & Purpose
3. Overall Description
4. Functional Requirements
5. Non-Functional Requirements
6. System Modules
7. Database Design Summary
8. API Overview
9. UML Diagrams Description
10. System Architecture
11. Security Design
12. UI/UX Guidelines
13. Deployment
14. Appendices

2. Introduction & Purpose

The School Management System (SMS) is a comprehensive digital platform designed to streamline administrative, academic, and operational processes of a school or educational institution. It supports students, teachers, parents, and administrators with modules for student management, attendance tracking, examination and grading, fee management, and a Lost & Found service.

2.1 Scope

- Web and mobile applications for different user roles
- Student information management and class assignment
- Teacher management and subject allocation
- Daily attendance recording and reporting
- Examination scheduling and result management
- Fee structure definition and payment tracking
- Lost & Found item reporting and claiming
- Role-based notifications and messaging
- Administrative dashboard and analytics

3. Overall Description

3.1 User Classes

Role	Description	Key Privileges
Student	Enrolled learners	View profile, attendance, results, fees, report lost/found items
Teacher	Teaching staff	Mark attendance, enter grades, view class lists
Parent	Guardians of students	View child’s attendance, results, fee status
Admin	School administrators	Manage users, classes, fees, moderate content
Super Admin	System owner	Full system configuration and role management

3.2 Operating Environment

- Mobile: Android 10+ / iOS 14+ (Flutter recommended)
- Web Admin: Modern browsers (Chrome, Edge, Firefox, Safari)
- Backend: Cloud or on-premise Linux servers
- Database: PostgreSQL 15+

4. Functional Requirements

ID	Module	Requirement	Priority
FR-01	Auth	User registration, login, password recovery, JWT sessions	High
FR-02	Students	Create, update, view student profiles and admission details	High
FR-03	Teachers	Manage teacher profiles and subject assignments	High
FR-04	Classes	Create classes/sections and assign class teachers	High
FR-05	Attendance	Mark daily attendance and generate reports	High
FR-06	Exams	Schedule exams, enter marks, generate report cards	High
FR-07	Fees	Define fee structures, record payments, track balances	High
FR-08	Lost & Found	Report lost/found items, search, claim process	Medium
FR-09	Notifications	Push and in-app notifications for key events	High
FR-10	Admin	Dashboard, user management, moderation, analytics	High
FR-11	Reports	Attendance, academic, and financial reports	Medium
FR-12	Messaging	Secure messaging related to claims or teacher-parent	Low

5. Non-Functional Requirements

Category	Requirement	Target
Performance	Page / API response time	< 2 seconds (95th percentile)
Performance	Concurrent users	Support ≥ 5,000
Security	Authentication	JWT + refresh tokens, Argon2/bcrypt
Security	Data in transit	TLS 1.2+
Reliability	Uptime	≥ 99.5%
Usability	Accessibility	WCAG 2.1 AA target
Scalability	Horizontal scaling	Stateless API services
Privacy	Data protection	Comply with local education data laws

6. System Modules

6.1 Core Academic Modules

Student Information System, Class & Section management, Subject allocation, Teacher assignment, Daily attendance with status (Present/Absent/Late/Excused), Examination scheduling, marks entry, automatic grade calculation and report cards.

6.2 Fees Module

Configurable fee structures per class/term, student fee assignment, payment recording (cash, bank, mobile money), balance tracking, overdue alerts, and payment history.

6.3 Lost & Found Module

Students and staff can report lost or found items with photos, search and filter items, submit claims with proof, and communicate securely. Admins moderate content.

6.4 Notifications & Communication

Role-based push notifications for attendance alerts, exam results, fee reminders, claim updates, and system announcements.

7. Database Design Summary

The system uses PostgreSQL with UUID primary keys. Major entities include: users, students, teachers, classes, subjects, class_subjects, attendance, exams, exam_results, fee_structures, student_fees, fee_payments, categories, locations, lost_found_items, claims, messages, notifications, and refresh_tokens.

A complete SQL migration script is provided separately (01_database_migration.sql) containing all CREATE TABLE statements, indexes, foreign keys, triggers, and seed data.

Key Relationships:

- User 1 — 0..1 Student / Teacher profile
- Class 1 — * Students
- Class + Subject — Teacher (class_subjects)
- Student — * Attendance records
- Exam — * Exam Results (per student)
- Student — * Fee records — * Payments
- User — * Lost & Found Items — * Claims — * Messages

8. API Overview

The system exposes a RESTful JSON API secured with JWT Bearer tokens. A complete OpenAPI 3.0 specification is provided in openapi.yaml.

Area	Key Endpoints
Auth	POST /auth/register, /login, /refresh, /forgot-password, /reset-password
Users	GET/PATCH /users/me
Students	GET/POST /students, GET/PATCH /students/{id}
Classes	GET/POST /classes, GET /classes/{id}/students
Attendance	POST /attendance, GET /attendance/student/{id}
Exams	GET/POST /exams, POST /exams/{id}/results
Fees	GET /fees/student/{id}, POST /fees/payments
Lost & Found	GET/POST /lost-found/items, POST .../claims, PATCH /claims/{id}
Notifications	GET /notifications, PATCH /notifications/read
Admin	GET /admin/stats + moderation endpoints

9. UML Diagrams Description

9.1 Use Case Diagram

Actors: Student, Teacher, Parent, Admin. Major use cases include Manage Profile, View Attendance, View Results, Pay Fees, Report Lost/Found Item, Claim Item, Mark Attendance, Enter Grades, Manage Users, and View Dashboard Analytics.

9.2 Core Class Diagram

Central classes: User, Student, Teacher, Class, Subject, Attendance, Exam, ExamResult, FeeStructure, StudentFee, Payment, LostFoundItem, Claim, Message, Notification. Enumerations are used for roles, attendance status, exam type, fee status, item type/status.

9.3 Key Sequence – Mark Attendance

Teacher opens class list → selects date → marks status per student → submits → API validates and stores records → optional notification to parents of absentees.

9.4 Item Lifecycle (Lost & Found)

Draft → Open (Lost/Found) → Under Claim → Claimed → Closed → Archived (after 90 days).

10. System Architecture

Layered architecture: Presentation (Flutter mobile + React admin), API layer (NestJS or FastAPI), Business services, Data access (PostgreSQL + Redis), and external services (FCM, S3-compatible storage, email).

Layer	Recommended Technology
Mobile App	Flutter 3.x
Admin Web	React + TypeScript
Backend API	NestJS or FastAPI
Database	PostgreSQL 16
Cache	Redis
File Storage	S3 / MinIO
Push Notifications	Firebase Cloud Messaging
Auth	JWT + Argon2

11. Security Design

- Passwords hashed with Argon2id or bcrypt (high work factor)
- Short-lived JWT access tokens + rotating refresh tokens
- Role-Based Access Control (RBAC) on every protected endpoint
- TLS 1.2+ for all communications
- Input validation, rate limiting, and account lockout
- Audit logging of administrative actions
- Principle of least privilege for database accounts

12. UI/UX Guidelines

Mobile-first design with clear role-based home screens. Students see attendance, results and fees at a glance. Teachers see today’s classes and quick attendance marking. Admins have a data-rich dashboard. Consistent color coding (e.g. green = present/paid, red = absent/overdue). Support for light/dark themes and accessibility standards.

Key Mobile Screens: Login, Student Dashboard, Attendance View, Results, Fee Status, Lost & Found Feed, Report Item, Claim, Profile.

Key Admin Screens: Dashboard, Students, Teachers, Classes, Attendance Reports, Exam Management, Fee Management, Lost & Found Moderation, Users.

13. Deployment Architecture

- Containerized backend (Docker) with horizontal scaling
- Managed PostgreSQL with automated backups
- Redis for caching and session support
- Object storage + CDN for images and documents
- CI/CD pipeline (GitHub Actions recommended)
- Environments: Development, Staging, Production
- Monitoring: error tracking (Sentry) + metrics

14. Appendices

14.1 Deliverables Accompanying this Document

- 01_database_migration.sql – Complete PostgreSQL schema
- openapi.yaml – Full OpenAPI 3.0 specification
- PlantUML / Mermaid sources for diagrams (available on request)

14.2 Future Enhancements

- Parent mobile app with real-time alerts
- Timetable / schedule management
- Library management module
- Transport / bus tracking
- Online admission portal
- AI-assisted result analysis and insights

End of Document

School Management System – SRS & Design Documentation – Version 1.0